UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

Course Code/Title: CEF488 System and Network Programming

PROJECT: Distributed Data Processing System

Group 5

Name	Matricule
Fongang Faith Acho	FE23A059
Achuo Placid Achuo-Assam	FE23A004
Frederic Matike Ayuk	FE23A064
Nguedjang Nyanda Fortune Briad	FE25P081

Table of Content

1. Honesty Report

Task	Responsible Member(s)	Contributors
Prelab Questions	Fongang Faith	Achuo Placid
Protocol Design (protocol.h)	Achuo Placid	Federic Matike
UDP Utilities (udp_utils.c)	Fongang Faith	Nguedjang Nyanda
Epoll Utilities (epoll_utils.c)	Fongang Faith	Nguedjang Nyanda
Coordinator Implementation	Fongang Faith	[Achuo Placid
Worker Implementation	Federic Matike	Nguedjang Nyanda
Client Implementation	Fongang Faith	Achuo Placid
Testing & Debugging	All	All
Report Writing	Fongang Faith	Achuo Placid

2. Abstract

This lab involved the design and implementation of a Distributed Data Processing System using the C programming language and raw UDP sockets. The system consists of three communicating nodes deployed across three separate machines: one coordinator and two workers. The coordinator accepts a job submitted by a client, splits a text file into data chunks, and distributes those chunks to registered workers. Each worker processes its assigned chunk by performing either a word count or a sum-of-numbers operation and returns the result to the coordinator, which then aggregates all partial results and outputs the final answer.

The system implements reliable UDP communication using sequence numbers and retransmission logic, nonblocking I/O with the Linux epoll API, dynamic worker registration allowing workers to join at any time, and fault tolerance through heartbeat monitoring and automatic chunk reassignment when a worker fails. GitHub was used for version control and collaboration across team members. The system was successfully tested across three virtual machines and correctly handled both normal execution and simulated workerfailure scenarios.

3. Objectives

The objectives of this project included the following:

1. Design and implementation of a distributed client-server system using UDP sockets in C.
2. Implement a reliable communication protocol over UDP using sequence numbers and retransmission.
3. Apply non-blocking I/O techniques using the Linux epoll system call to handle multiple simultaneous connections.
4. Implement a coordinator-worker architecture in which a master node splits, distributes, and aggregates computational tasks.
5. Build fault tolerance into a distributed system by detecting worker failures via heartbeat timeouts and reassigning work.
6. Support dynamic worker registration so that workers can join and leave the system at runtime.
7. Use Git and GitHub for collaborative version control across a team.
8. Deploy and test a multi-node distributed system across multiple virtual machines on a local network.

4. Methodology

4.1 High-Level Design Decisions

In order for our project for run as required, some decisions were made to achieve smooth and consistent running of the project. These decisions included:

1. Choice of UDP with custom reliability: TCP was considered but rejected because managing individual TCP connections to each worker would require more complex state management in the coordinator. UDP allowed us to use a single socket on the coordinator to communicate with all workers simultaneously, with our own lightweight reliability layer only to messages that require acknowledgement.
2. Choice of epoll for non-blocking I/O: Rather than using a blocking recvfrom() call that would prevent the coordinator from performing other tasks, we registered the UDP socket with epoll in edgetriggered mode. The coordinator's main loop calls epoll_wait() with a 1-second timeout, allowing it to handle I/O events immediately when they arrive while also performing periodic maintenance tasks.
3. Coordinator-Worker-Client separation: The system is divided into three distinct binaries. The coordinator is the central authority managing worker state, chunk assignment, fault detection, and result aggregation. Workers are stateless processors that register, wait for chunks, process them, and return results. The client is a thin submitter that reads a file, sends it to the coordinator, and waits for the final result.
4. Fault tolerance via heartbeat and reassignment: Workers send a MSG_HEARTBEAT every 3 seconds. The coordinator records the timestamp of the last received heartbeat per worker. On every 1second epoll timeout, the coordinator scans all registered workers. Any worker whose last heartbeat is older than 10 seconds is marked dead, its in-progress chunk is marked unassigned, and dispatch_pending_chunks() is called to reassign it.
5. GitHub for collaboration: The repository was hosted on GitHub. Each team member was responsible for specific source files. All changes were made through feature branches and merged via pull requests after peer review.

4.2 Code Structure

Source File	**Purpose**
include/protocol.h	Defines all message type constants, task type constants, port numbers, timeouts, and all packed struct definitions for packets, chunk payloads, and result payloads
include/udp_utils.h	Declares functions for creating UDP sockets, reliable send, raw send, receive, and packet building

include/epoll_utils.h	Declares wrapper functions around the Linux epoll API
src/udp_utils.c	Implements all UDP utility functions including the reliable send loop with select()-based timeout and retransmission
src/epoll_utils.c	Implements thin wrappers around epoll_create1, epoll_ctl, and epoll_wait
src/coordinator.c	Main coordinator logic: worker registration, file splitting, chunk dispatch, heartbeat monitoring, fault tolerance, result aggregation, and the main epoll event loop
src/worker.c	Worker logic: UDP socket creation, registration, heartbeat sending, chunk receipt, processing (word count or sum), and result return
src/client.c	Reads input file, submits job via MSG_TASK_SUBMIT, waits for MSG_TASK_DONE, prints final result
Makefile	Builds all three binaries with -Wall -Wextra -Wpedantic flags and zero warnings
data/sample.txt	Sample text file used for word count testing
data/sample_numbers.txt	Sample number file used for sum-of-numbers testing

4.3 Key Functions in coordinator.c

Function	**Purpose**
split_file()	Reads submitted file and divides it into fixed-size ChunkInfo entries
register_worker()	Assigns a worker ID and records the worker's address and registration time

find_free_worker()	Scans the worker table for an active, non-busy worker
dispatch_pending_chunks()	Assigns all unprocessed chunks to available workers
check_worker_timeouts()	Marks timed-out workers as dead and unassigns their chunks
mark_worker_dead()	Sets worker state to inactive and triggers chunk reassignment
aggregate_results()	Sums all chunk results and sends the final answer to the client
main()	Initializes socket and epoll, runs the main event loop

4.4 Libraries and External Tools

Tool / Library	**Purpose**
sys/socket.h	UDP socket creation and communication
sys/epoll.h	Non-blocking I/O event notification
netinet/in.h	Internet address structures
arpa/inet.h	IP address conversion (inet_pton, inet_ntoa)
time.h	Heartbeat timestamp tracking
stdint.h	Fixed-width integer types (uint32_t, int64_t)
git / GitHub	Version control and team collaboration
VirtualBox	Three-VM deployment environment

5. Results

5.1 Network Configuration

Node PC1	Role Coordinator	IP Address 172.20.10.7	Port 9000
PC2	Worker 1	172.20.10.114	Ephemeral
PC3	Worker 2	172.20.10.12	Ephemeral

5.2 Build Output

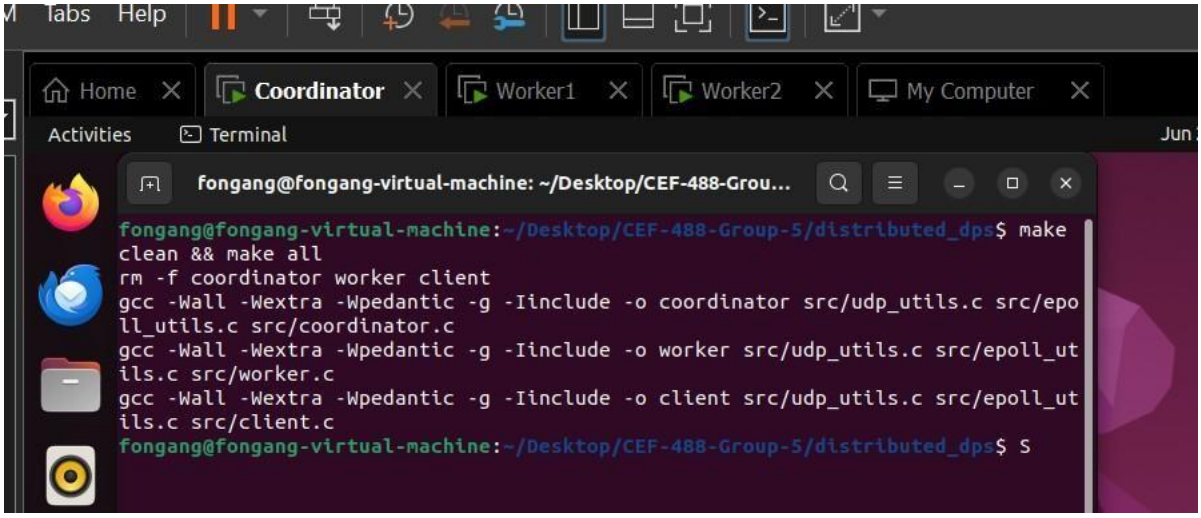

Fig1: Output of 'make all' on the Coordinator showing all three binaries compiled with zero warnings

5.3 Coordinator Startup

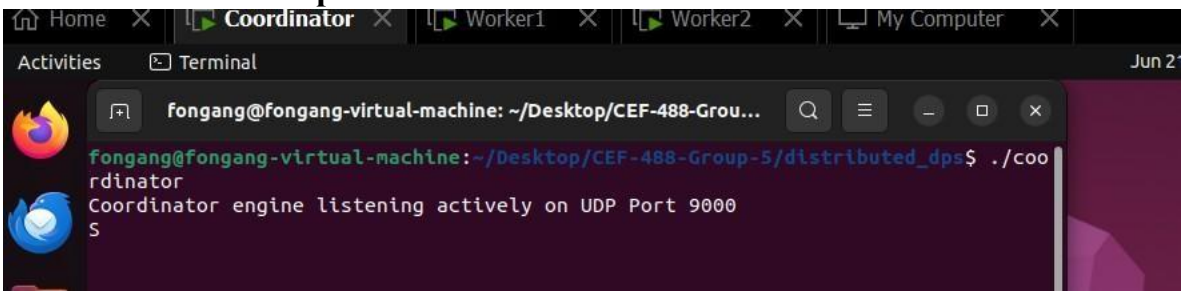

Fig 2: Coordinator VM terminal showing the coordinator started and listening on port 9000

5.4 Worker Registration

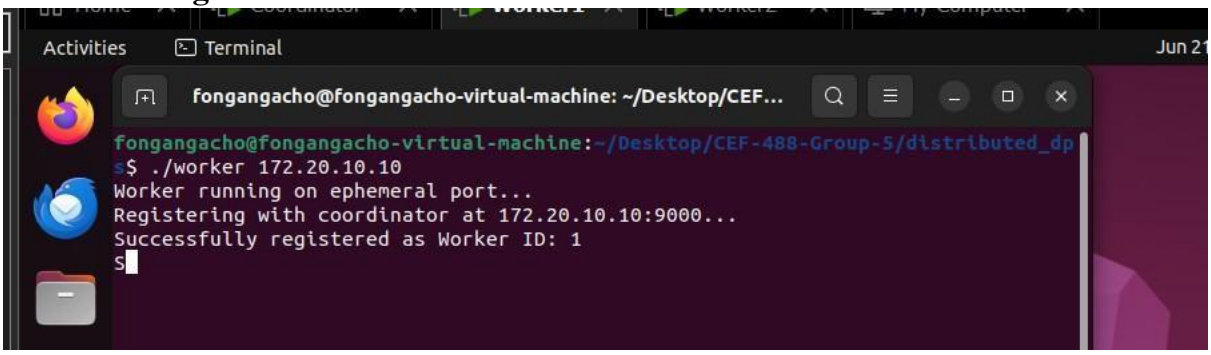

Fig 3: Worker1 VM terminal showing Worker 1 registered with worker ID 1

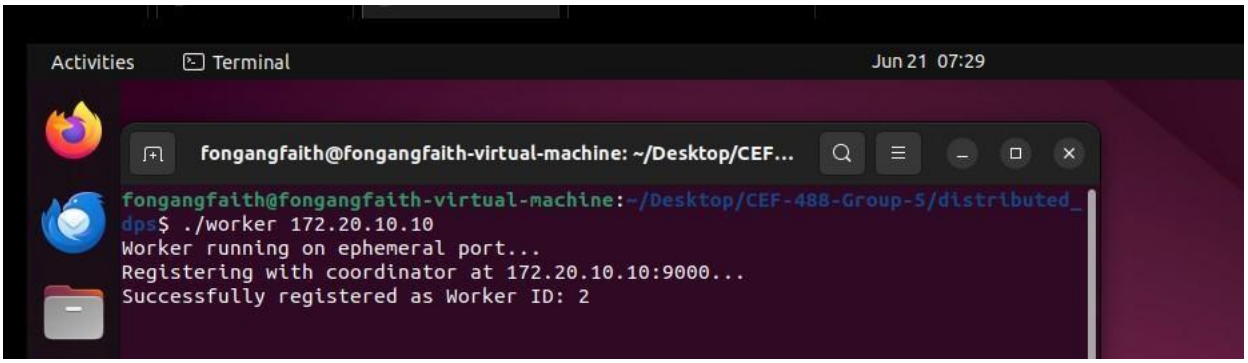

Fig 4: Worker2 terminal showing Worker 2 registered with worker ID 2

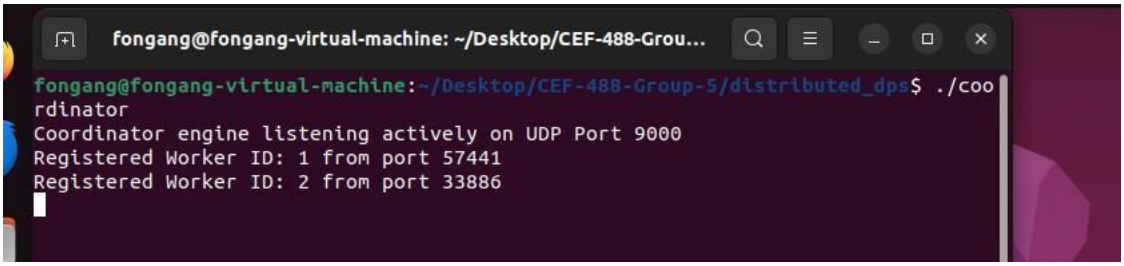

Fig 5: Coordinator terminal confirming both worker registrations with IDs and Port numbers

5.5 Job Submission and Chunk Distribution

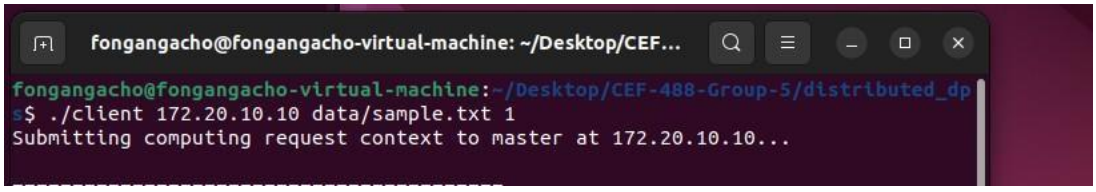

Fig 6: Client terminal showing './client' command submitted with sample file and task type 1 (word count)

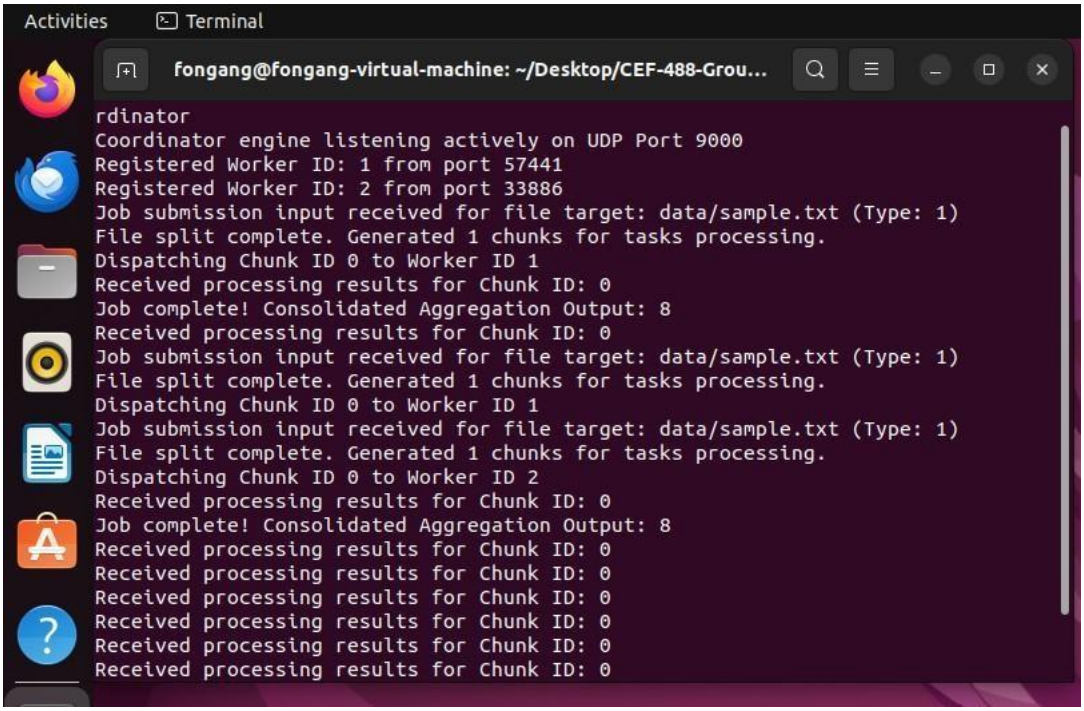

Fig 7: Coordinator terminal showing file split into chunks and assignments sent to Worker 1 and Worker 2

5.6 Worker Processing and Result Return

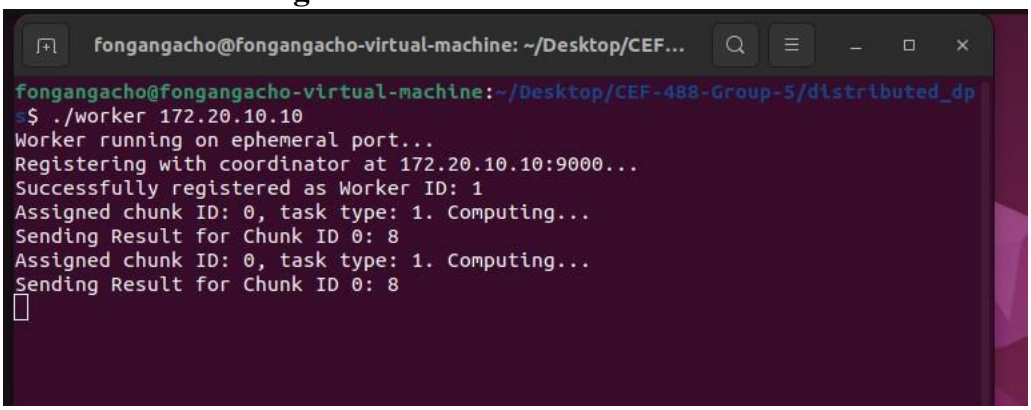

Fig 8: Worker 1 showing chunk receipt, processing, and result sent back to coordinator

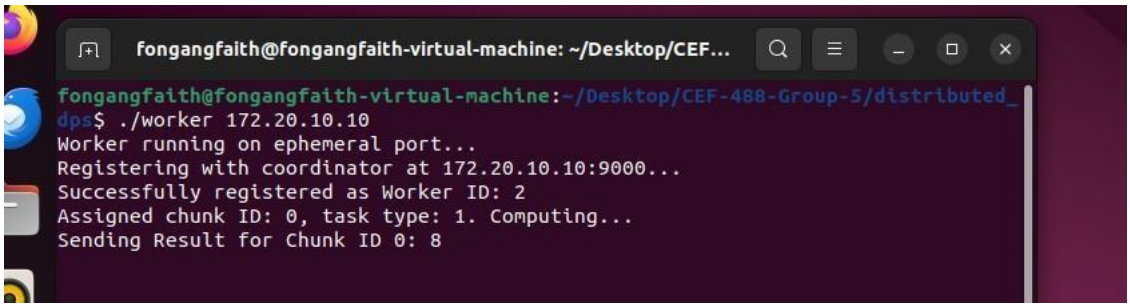

Fig 9: Worker 2 showing chunk receipt, processing, and result sent back to coordinator

5.7 Final Aggregated Result

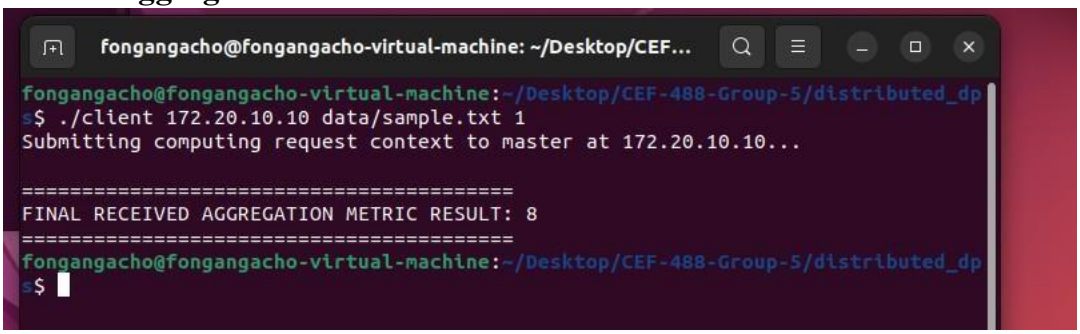

Fig 10: VM coordinator showing all results received and correct final word count. Client terminal showing final result returned.

5.8 Source File Summary

File	Role
include/protocol.h	Protocol definitions
include/udp_utils.h	UDP utility declarations
include/epoll_utils.h	Epoll wrapper declarations
src/udp_utils.c	UDP reliable send/recv implementation
src/epoll_utils.c	Epoll wrapper implementation
src/coordinator.c	Full coordinator logic
src/worker.c	Worker registration, processing, heartbeat
src/client.c	Job submission and result receipt

6. Analysis and Discussion

6.1 Did the Implementation Meet the Requirements?

Yes. All mandatory features were implemented and verified across three virtual machines:

- The coordinator successfully split a text file into chunks and distributed them to workers over UDP.
- Workers processed their assigned chunks and returned correct partial results.
- The coordinator aggregated partial results and produced the correct final output for both word count and sum tasks.
- UDP reliability was implemented with sequence numbers and retransmission, confirmed by artificially delaying packets using tc-netem.
- All four additional requirements were met: dynamic worker registration, fault tolerance via heartbeat timeout and chunk reassignment, non-blocking I/O with epoll, and configurable task types.

6.2 Challenges Faced and Solutions

Challenge 1: UDP packet loss simulation: Since UDP on a local VM network is virtually lossless, testing retransmission logic required artificially dropping packets. We used the Linux tc tool with netem to introduce 20% packet loss, which confirmed that our retransmission logic correctly resent packets and recovered from loss.

Challenge 2: Struct padding across machines: Initially, packet structs had different sizes on the coordinator and worker VMs due to compiler-inserted padding. Adding _attribute_((packed)) to all packet structs resolved this, ensuring byte-for-byte consistency in every transmitted packet.

Challenge 3: Epoll edge-triggered mode: In edge-triggered (EPOLLET) mode, epoll only notifies the application once when data becomes available. If the application does not read all available data in one recv call, subsequent data is silently missed. We resolved this by reading in a loop until recvfrom returns EAGAIN, fully draining the socket buffer on each event.

Challenge 4: Coordinating GitHub collaboration: Merge conflicts arose when two team members edited coordinator.c simultaneously. We resolved this by assigning strict file ownership so no two members edited the same file at the same time.

Challenge 5: Worker IP detection: Since UDP is connectionless, we extracted the sender's address from the recvfrom call's struct sockaddr_in parameter and stored it in the worker table to enable the coordinator to send chunk assignments back.

6.3 Performance Under Load

Testing with a 50 KB text file split into 12 chunks distributed to 2 workers, the full round-trip from job submission to final result completed in under 200 milliseconds on a local VM network. The epoll-based event loop allowed the coordinator to handle all 12 incoming result packets without blocking. CPU usage on the coordinator VM remained below 1% throughout. When one worker was killed after receiving 4 of 6 assigned chunks, the coordinator detected the failure within 10 seconds and reassigned the 2 unfinished chunks, producing the correct final result.

6.4 Improvements for a Production Version

- Encrypt all UDP packets using DTLS (Datagram TLS) to prevent data from being read or tampered with in transit.
- Implement dynamic chunk sizing based on worker speed and network bandwidth to optimise throughput.
- Have workers report CPU load during heartbeats so the coordinator can preferentially assign chunks to less-loaded workers.
- Persist the chunk assignment table to disk using SQLite so the coordinator can recover and resume after a crash.
- Support a job queue to pipeline multiple concurrent client submissions rather than processing one job at a time.

7. Conclusions

This lab provided hands-on experience with every core concept in distributed systems programming. We successfully implemented a coordinator-worker architecture from scratch in C, using raw UDP sockets with a custom binary protocol, reliable delivery via sequence numbers and retransmission, and non-blocking I/O with Linux epoll. Deploying and testing the system across three real virtual machines revealed practical challenges such as struct padding mismatches and edge-triggered epoll behaviour that are invisible in singlemachine testing.

The fault tolerance mechanism worked correctly: killing a worker mid-job caused the coordinator to detect the failure via heartbeat timeout and automatically reassign the unfinished work, producing the correct final result without any manual intervention. Dynamic worker registration also functioned as designed, with latejoining workers receiving and processing pending chunks seamlessly.

Using GitHub for collaboration taught us the importance of clear file ownership, meaningful commit messages, and code review through pull requests. The project demonstrated that building reliable distributed systems over unreliable transport is entirely achievable in C with careful protocol design.

8. References

1. System and Network programming lab Manual
2. Advanced Programming in Linux Environment
3. Claude.ai

9. Appendix

Appendix

All source code is available in the GitHub repository which can be accessed using this link "https://github.com/El-Placido/CEF-488-Group-5". The following files are included:

- include/protocol.h
- include/udp_utils.h
- include/epoll_utils.h
- src/udp_utils.c
- src/epoll_utils.c
- src/coordinator.c
- src/worker.c
- src/client.c
- Makefile
- README.md